

TASK 9

Deploying a Simple Node.js Application on AWS Elastic Beanstalk

Umar Satti

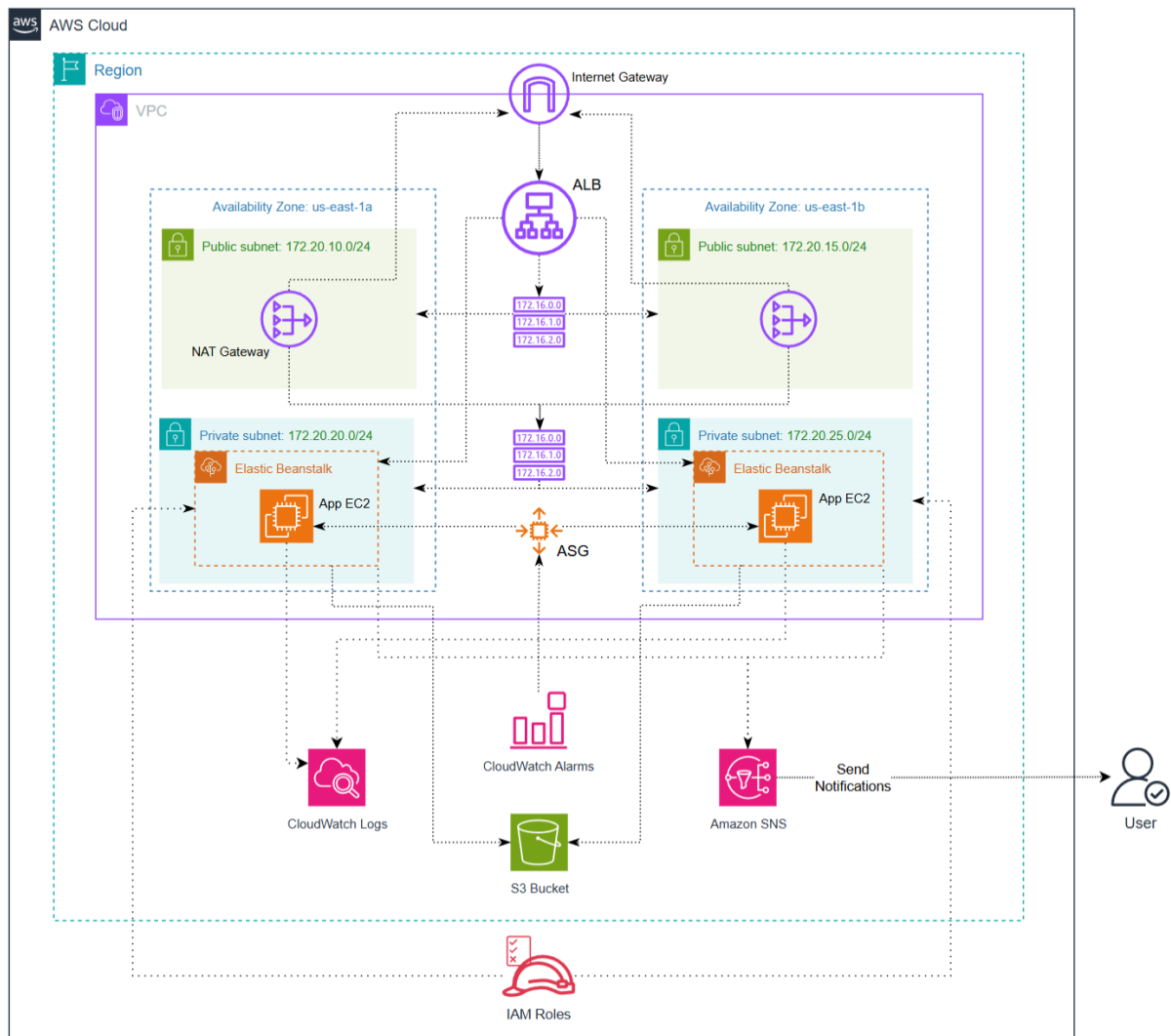
Table of Contents

Task Description.....	3
Architecture Diagram	3
Task 1: VPC Configuration	4
Task 1.1: Create VPC.....	4
Task 1.2: Create Internet Gateway	4
Task 1.3: Create NAT Gateway	5
Task 1.4: Create Subnets	6
Task 1.5: Create Route Tables	7
Task 1.6: Setting up Routes for Route tables	8
Task 1.7: Route Table Association.....	9
Task 1.8: Security Groups.....	10
Task 2: Elastic Beanstalk.....	12
Task 2.1: Create Elastic Beanstalk application	12
Task 2.2: IAM Roles and Permissions	12
Task 2.2.1: IAM Service Role for Elastic Beanstalk	12
Task 2.2.2: IAM Role for EC2 Instances	13
Task 2.3: Configure an Elastic Beanstalk Environment.....	14
Step 1: Configure environment.....	14
Step 2: Configure service access.....	15
Step 3: Set up Networking, database, and tags	15
Step 5: Configure updates, monitoring, and logging	18
Step 6: Review	19
Task 3: Verify Infrastructure Deployment.....	21
Verify EC2 Instances are launched	21
Verify Target Groups	21
Verify Autoscaling Group.....	21
Verify ALB is working	22
Amazon SNS.....	23
CloudWatch Log groups	24
Environment Health	25
CloudWatch Alarms	25
CloudWatch Alarm Conditions.....	26
Using CloudWatch Alarms	26
Task 4: Update Application Code and Redeploy.....	29
4.1 Prepare Updated Application Code	29
4.2 Upload Updated Code in Elastic Beanstalk.....	29
4.3 Validate the Deployment.....	30
Task 5: Clean Up	31
Task 6: Troubleshooting.....	33

Task Description

This task involves deploying a Node.js application to AWS Elastic Beanstalk using the AWS Management Console. It includes creating the application and environment, uploading and deploying application code, configuring platform and scaling settings, enabling monitoring and health checks, and validating the deployment through Elastic Beanstalk endpoints and CloudWatch metrics.

Architecture Diagram



Task 1: VPC Configuration

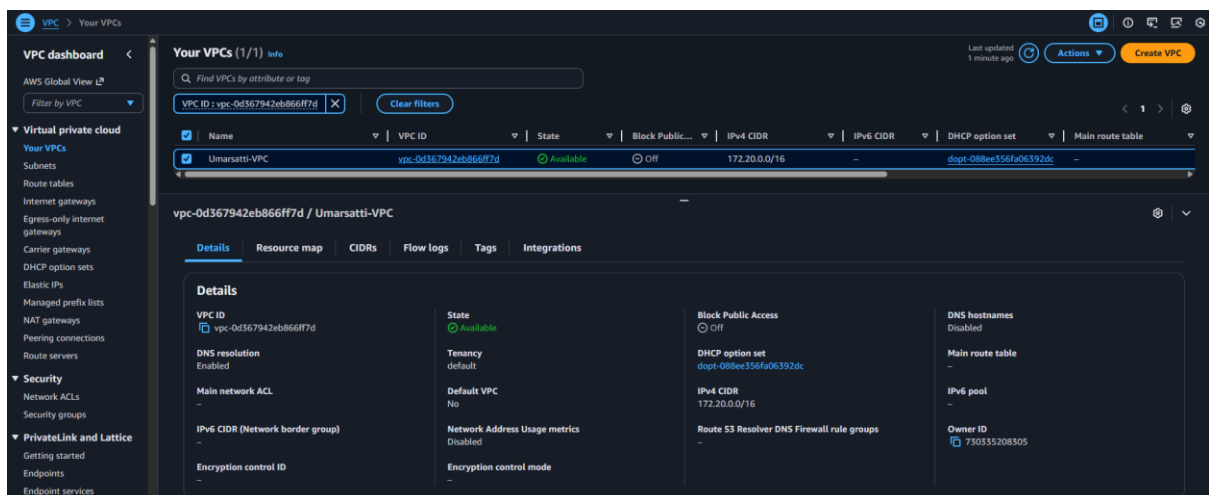
Task 1.1: Create VPC

A **Virtual Private Cloud (VPC)** is a logically isolated network environment within AWS, allowing the creation of secure and private infrastructure.

Steps:

1. Sign in to the AWS Management Console.
2. Navigate to VPC service using the search bar at the top.
3. In the VPC Console, click **Your VPCs**.
4. Click **Create VPC** button on the top right.
5. Choose **VPC only** option for 'Resources to create'.
6. Choose a name for the VPC and add an IPv4 CIDR block.
7. Leave the **Tenancy** as **Default**.

As shown in the image below, under **Details** tab, a VPC has been created with the following configuration:



- Name: **Umarsatti-VPC**
- IPv4 CIDR range: **172.20.0.0/16**
- VPC ID: **vpc-0d367942eb866ff7d**

Task 1.2: Create Internet Gateway

An **Internet Gateway (IGW)** enables communication between instances in the VPC and the public internet.

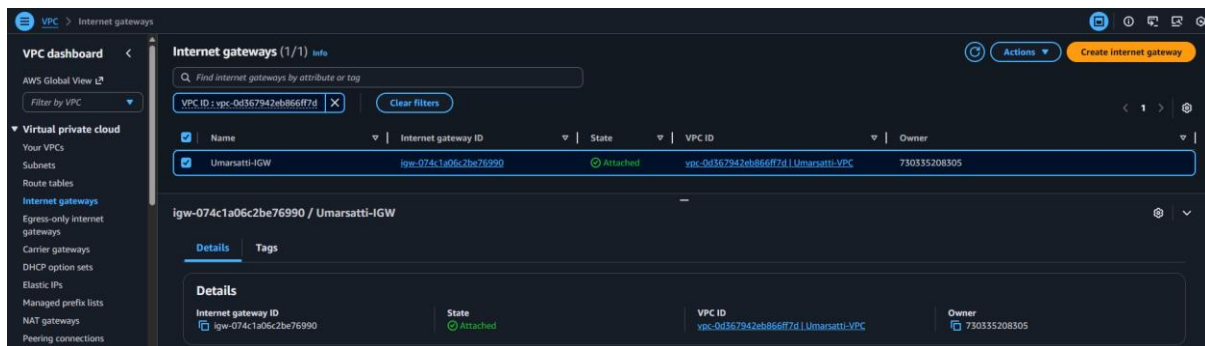
Steps:

1. Within the VPC Console, click **Internet gateways**.
2. Click on **Create internet gateway** button.

3. Choose a **Name** and add optional Tags.
4. Click **Create internet gateway** button. This creates an Internet gateway outside the VPC, and it needs to attach to the VPC created earlier.
5. Select **Actions** on the top right and click **Attach to VPC**. Choose the VPC ID (created earlier) and click **Attach internet gateway**. This attaches the Internet gateway to the VPC, allowing internet access.

As shown in the image below under **Details** tab, an internet gateway has been created with the following configuration:

- Name: **Umarsatti-IGW**
- State: **Attached**
- VPC ID: **vpc-0d367942eb866ff7d**
- Internet gateway ID: **igw-074c1a06c2be76990**



Task 1.3: Create NAT Gateway

A NAT Gateway (Network Address Translation Gateway) provides outbound internet access to private instances without exposing them to inbound traffic.

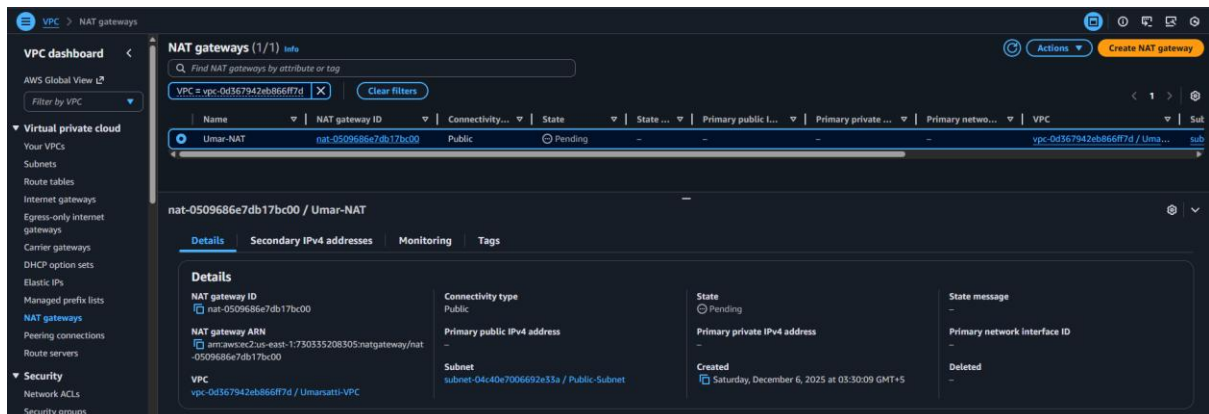
Steps:

1. Within the VPC Console, click **NAT Gateways**.
2. Click on **Create NAT gateway** button on the top right.
3. Choose **Availability mode** as **Zonal**. This helps attach NAT gateway in public subnet.
4. Choose a **Name**, **Subnet**, **Connectivity type**, **Elastic IP allocation ID**, and optional **Tags**.
5. When done, click on **Create NAT gateway** button to create the resource.

As shown in the image below under **NAT gateways** section, a NAT gateway is created with the following configuration:

- Name: **Umar-NAT**
- NAT gateway ID: **nat-0509686e7db17bc00**
- Connectivity type: **Public**
- VPC: **Umarsatti-VPC**

Note: When creating the NAT Gateway, click on **Allocate Elastic IP** option. This automatically generates an **Elastic IP** for NAT Gateway resource.



NAT Gateway is set up in the public subnet and assigned an Elastic IP. The private EC2 instances can communicate externally without being directly reachable from the internet. This maintains a secure boundary for private subnets.

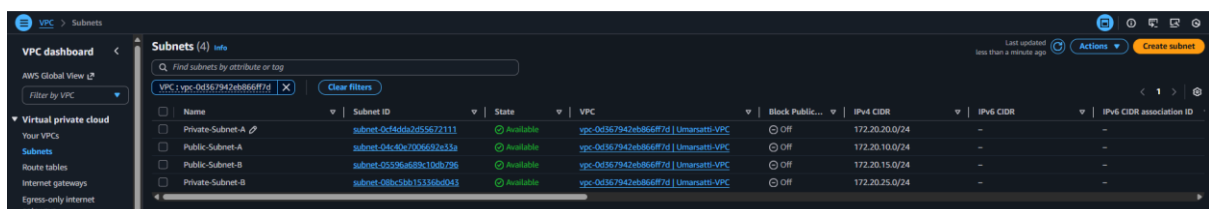
Note: For high availability and fault tolerance, create a NAT gateway in each public subnet. For this project, only a single NAT gateway has been deployed in Public Subnet A (but diagram indicates two).

Task 1.4: Create Subnets

Steps:

1. Within the VPC Console, click **Subnets**.
2. Click on **Create subnet** button.
3. Choose a **Subnet name**, **Availability Zone**, **IPv4 VPC CIDR block**, **IPv4 subnet CIDR Block** and optional **Tags**.
4. Repeat this process by clicking on **Add new subnet** button at the bottom to add more subnets (Public and Private).
5. After adding the required subnets, click on **Create subnet** button at the bottom right.

As shown in the image below under **Subnets** section, four subnets have been created with the following configuration:



Public Subnet in us-east-1a:

- Name: **Public-Subnet-A**
- IPv4 CIDR: **172.20.10.0/24**

Public Subnet us-east-1b:

- Name: **Public-Subnet-A**
- IPv4 CIDR: **172.20.15.0/24**

Private Subnet us-east-1a:

- Name: **Private-Subnet-A**
- IPv4 CIDR: **172.20.20.0/24**

Private Subnet us-east-1b:

- Name: **Private-Subnet-B**
- IPv4 CIDR: **172.20.25.0/24**

In this design:

- The **Public Subnet** hosts internet-facing resources (e.g., Load Balancer).
- The **Private Subnets** host internal EC2 instances running via Elastic Beanstalk and Auto Scaling Group.

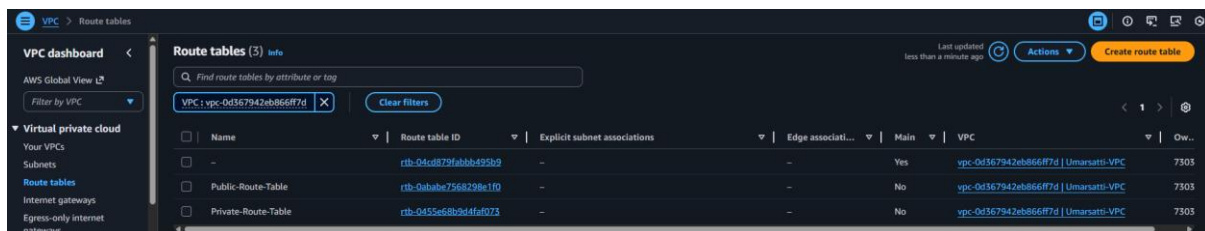
Task 1.5: Create Route Tables

Route Tables define how network traffic moves within the VPC. Each route table directs traffic either internally (within the VPC) or externally (to the internet or a NAT gateway).

Steps:

1. Within the VPC Console, click **Route tables**.
2. Click on **Create route table** button on the top right.
3. Choose a **Name**, **VPC**, and optional **Tags**.
4. When done, click on **Create route table** button to create the resource.

As shown in the image below under **Route tables** section, two route tables have been created (excluding default route table) with the following configuration:



	Name	Route table ID	Explicit subnet associations	Edge associati...	Main	VPC	Own...
☐	-	rtb-04cd879fabbb495b9	-	-	Yes	vpc-0d367942eb866f7d Umarsati-VPC	7303
☒	Public-Route-Table	rtb-0ababe7568298e1f0	-	-	No	vpc-0d367942eb866f7d Umarsati-VPC	7303
☒	Private-Route-Table	rtb-0455e68b9d4fa073	-	-	No	vpc-0d367942eb866f7d Umarsati-VPC	7303

Public Route table:

- Name: **Public-Route-Table**
- Route table ID: **rtb-0ababe7568298e1f0**

Private Route table:

- Name: **Public-Route-Table**
- Route table ID: **rtb-0455e68b9d4faf073**

In this configuration:

- The **Public Route Table** directs internet-bound traffic to the **Internet Gateway**.
- The **Private Route Table** forwards internet-bound requests through the **NAT Gateway** (set up in Task 1.3), maintaining private subnet isolation.

This ensures clear routing logic between internal and external networks.

Task 1.6: Setting up Routes for Route tables

Steps:

1. Within the Route tables console, select a Route table (example: Public-Route-Table) and click **Edit routes** on the right.
2. Add a new route by choosing a **Destination** and **Target**.
3. Click **Save changes**. This adds a route to this route table.

Public Routes (Public-Route-Table):

- Destination: **0.0.0.0/0**
- Target: **Umarsatti-IGW | igw-074c1a06c2be76990** (Internet gateway)

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-074c1a06c2be76990	Active	No	Create Route
172.20.0.0/16	local	Active	No	Create Route Table

Private Routes (Umar-Pr-Rt):

- Destination: **0.0.0.0/0**
- Target: **Umar-NAT | nat-0509686e7db17bc00** (NAT gateway)

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-0509686e7db17bc00	Active	No	Create Route
172.20.0.0/16	local	Active	No	Create Route Table

This step defines how traffic flows between subnets and external networks:

- The **Public route table** sends all internet-bound traffic (0.0.0.0/0) to the **Internet Gateway**, enabling access for public resources like the load balancer DNS.

- The **Private route table** provides internet-bound traffic through the **NAT Gateway**, allowing private EC2 instances to initiate outbound connections without exposure to the internet.

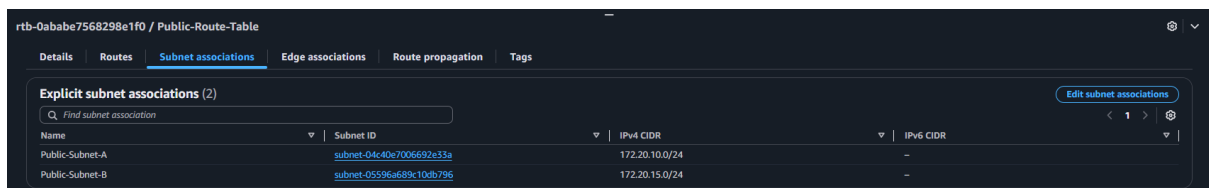
Task 1.7: Route Table Association

Steps:

1. Within the Route tables console, select a Route table (example: Public-Route-Table) and click **Subnet associations** tab under the details section.
2. Click **Edit subnet associations** and select the specific subnet (e.g. nginx-public-subnet)
3. Click **Save associations** button on the bottom right.

Public Route table Subnet associations

- Names: **Public-Route-Table**
- Subnets: **Public-Subnet-A** and **Public-Subnet-B**
- IPv4 CIDR: **172.20.10.0/24** and **172.20.15.0/24**

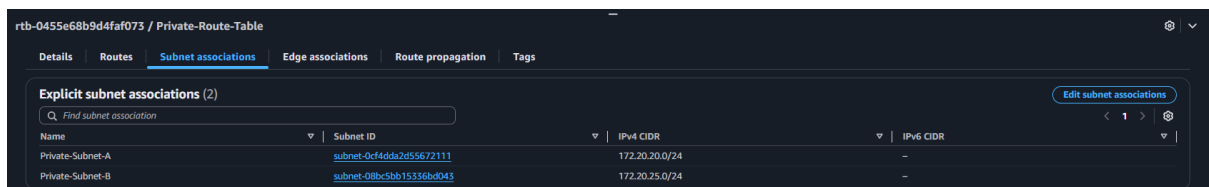


The screenshot shows the 'Subnet associations' tab for the 'Public-Route-Table' (rtb-0a3abe7568298e1f0). It displays two explicit subnet associations in a table.

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
Public-Subnet-A	subnet-04c40e7006692e33a	172.20.10.0/24	-
Public-Subnet-B	subnet-05596a689c10db796	172.20.15.0/24	-

Private route table Subnet Associations

- Names: **Private-Route-Table**
- Subnet IDs: **Private-Subnet-A** and **Private-Subnet-B**
- IPv4 CIDR: **172.20.20.0/24** and **172.20.25.0/24**



The screenshot shows the 'Subnet associations' tab for the 'Private-Route-Table' (rtb-0455e68b9d4fa073). It displays two explicit subnet associations in a table.

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
Private-Subnet-A	subnet-0cf4d4da2d55672111	172.20.20.0/24	-
Private-Subnet-B	subnet-08bc5bb15336bd043	172.20.25.0/24	-

Each subnet must be linked to a route table to control its network traffic.
Associating subnets ensures:

- **Public Subnet** uses the Internet Gateway for external communication.
- **Private Subnets** use the NAT Gateway for secure outbound traffic.

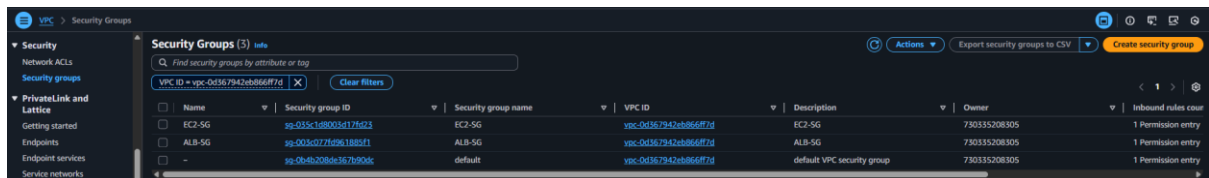
Without this association, subnets would rely on the default main route table, potentially misrouting traffic.

Task 1.8: Security Groups

Security Groups act as virtual firewalls that regulate inbound and outbound traffic at the instance level.

Steps:

1. Within the VPC Console, click **Security Groups**.
2. Click on **Create security gateway** button on the top right.
3. Choose a **Security group name**, **Description**, and **VPC** (Umarsatti-VPC).
4. Add Inbound rules by selecting **Type**, **Protocol**, **Port range**, **Source**, and **Description** (optional).
5. Leave Outbound rules as default (All traffic).

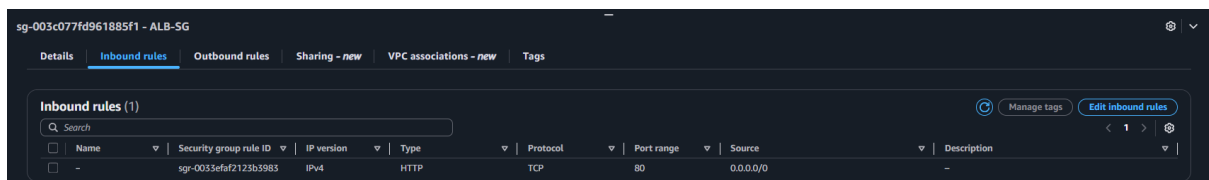


Task 1.8.1: Application Load Balancer Security group (ALB-SG)

Here is the configuration for the public subnet security group:

- Security group name: **ALB-SG**
- Security group ID: **sg-003c077fd961885f1**
- Inbound Rules:

Type	Protocol	Port range	Source
HTTP	TCP	80	0.0.0.0/0



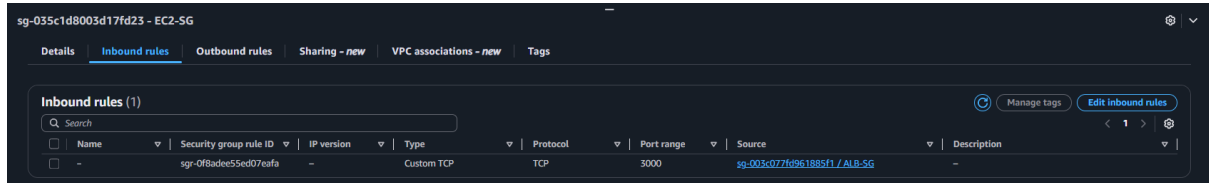
Task 1.8.2: EC2 Instance Security group (EC2-SG)

Here is the configuration for the public subnet security group:

- Security group name: **EC2-SG**
- Security group ID: **sg-035c1d8003d17fd23**

- Inbound Rules:

Type	Protocol	Port range	Source
Custom TCP	TCP	5000	ALB-SG



In this configuration:

- **ALB-SG:** Allows HTTP (port 80) from any external source.
- **EC2-SG:** Allows TCP port 5000 traffic from ALB-SG.

This layered security model follows the principle of least privilege, ensuring only the load balancer can reach private EC2 instances

Defining a custom VPC (e.g., using CIDR 172.20.0.0/16) provides control over the IP address range, subnets, routing, and security configurations. This VPC serves as the foundational network for deploying both public and private resources while maintaining clear separation and security boundaries.

Task 2: Elastic Beanstalk

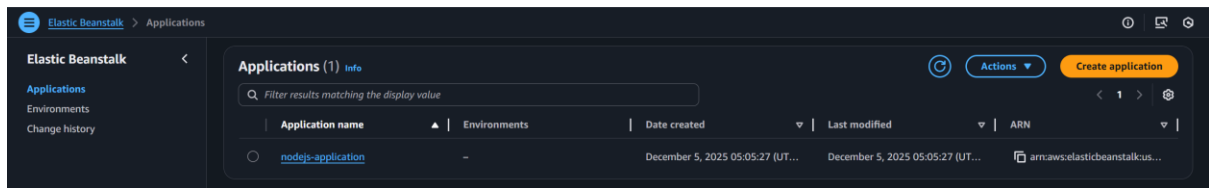
Elastic Beanstalk is used to deploy web applications into the AWS Cloud on a variety of supported platforms. Users build and deploy their applications. Elastic Beanstalk provisions Amazon EC2 instances, configures load balancing, sets up health monitoring, and dynamically scales the environment.

Task 2.1: Create Elastic Beanstalk application

Steps:

1. Sign in to the AWS Management Console.
2. Navigate to **Elastic Beanstalk** service using the search bar at the top.
3. In the Elastic Beanstalk Console, click **Applications** on the left navigation bar.
4. Click **Create application** button on the top right.
5. Choose **Application name**, **Description**, and **Tags**. Click **Create** button after to create this resource.

As shown in the image below, an Elastic Beanstalk application has been created with the name **nodejs-application**.



Task 2.2: IAM Roles and Permissions

Task 2.2.1: IAM Service Role for Elastic Beanstalk

- Role name: aws-elasticbeanstalk-ec2-role
- IAM Policies attached:
 - AWSElasticBeanstalkEnhancedHealth
 - AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy

Step 1: Select trusted entity

Select trusted entity info

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
Elastic Beanstalk

Choose a use case for the specified service.

Use case

- ☐ Elastic Beanstalk - Compute
Allows your environment's EC2 instances to perform operations required for your application.
- ☒ **Elastic Beanstalk - Environment**
Allows access to other AWS service resources that are required to create and manage environments.

Cancel Next

Step 2: Add permissions

Add permissions info

Permissions policies (2) info

The type of role that you selected requires the following policy.

Policy name	Type
AWSElasticBeanstalkEnhancedHealth	AWS managed
AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy	AWS managed

Set permissions boundary - optional

Cancel Previous Next

Task 2.2.2: IAM Role for EC2 Instances

- Role name: aws-elasticbeanstalk-ec2-role
- Add permissions
 - AWSElasticBeanstalkMulticontainerDocker
 - AWSElasticBeanstalkWebTier
 - AWSElasticBeanstalkWorkerTier
 - AmazonSSMManagedInstanceCore

Identity and Access Management (IAM)

aws-elasticbeanstalk-ec2-role info Delete

Allows your environment's EC2 instances to perform operations required for your application.

Summary Edit

Creation date December 05, 2025, 16:59 (UTC+05:00)	ARN arn:aws:iam::730335208305:role/aws-elasticbeanstalk-ec2-role	Instance profile ARN arn:aws:iam::730335208305:instance-profile/aws-elasticbeanstalk-ec2-role
Last activity -	Maximum session duration 1 hour	

Permissions Trust relationships Tags Last Accessed Revoke sessions

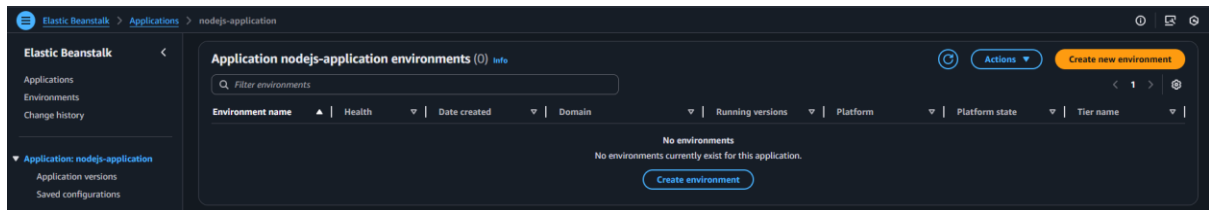
Permissions policies (4) info Simulate Remove Add permissions

You can attach up to 10 managed policies.

Policy name	Type	Attached entities
AmazonSSMManagedInstanceCore	AWS managed	4
AWSElasticBeanstalkMulticontainerDocker	AWS managed	1
AWSElasticBeanstalkWebTier	AWS managed	1
AWSElasticBeanstalkWorkerTier	AWS managed	1

Task 2.3: Configure an Elastic Beanstalk Environment

1. In the Elastic Beanstalk Console, click **Environments** on the left navigation bar.
2. Click **Create environment** button on the top right.
3. Follow along and complete each step for custom configuration.



Step 1: Configure environment

1. Environment tier
 - Select **Web server environment** (This is for apps accessed over HTTP)
2. Application information
 - nodejs-application (added by default)
3. Environment information
 - **Environment name:** nodejs-env
 - **Domain:** umarsatti-nodejs-app.us-east-1.elasticbeanstalk.com (or leave it blank)
 - **Description:** Environment for running simple Node.js application
4. Platform Configuration
 - **Platform:** Node.js
 - **Branch:** Node.js 22 running on 64-bit Amazon Linux 2023
 - Should match the version defined inside *package.json* file
 - Elastic Beanstalk updates it accordingly
 - **Version:** 6.7.0
5. Application code
 - Upload your code
 - Select **Local file**
 - Choose Node.js project .zip file from local computer
 - Make sure the project files are zipped/compressed prior
 - File: nodejs-app.zip
 - **Contains:** package.json, index.html, and app.js

Application code [Info](#)

- ☐ Sample application
- ☐ Existing version
Application versions that you have uploaded.
- ☒ Upload your code
Upload a source bundle from your computer or copy one from Amazon S3.

Version label
Unique name for this version of your application code.

1

Source code origin. Maximum size 500 MB

- ☒ Local file
- ☐ Upload application
 Upload application
 Choose file
 File name: nodejs-app.zip
 File must be less than 500MB max file size
- ☐ Public S3 URL

6. Presets

- Configuration presets:
 - Select **High availability** (for use with Load balancer)

7. Click **Next** to proceed

Step 2: Configure service access

Elastic Beanstalk > Create environment

Step 1: Configure environment

Step 2: **Configure service access**

Step 3 - optional: Set up networking, database, and tags

Step 4 - optional: Configure instance traffic and scaling

Step 5 - optional: Configure updates, monitoring, and logging

Step 6: Review

Configure service access [Info](#)

Service access
IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create and manage your environment. Both the IAM role and instance profile must be attached to IAM managed policies that contain the required permissions. [Learn more](#)

Service role
Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.

aws-elasticbeanstalk-service-role [Create role](#)

EC2 instance profile
Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

aws-elasticbeanstalk-ec2-role [Create role](#)

EC2 key pair - optional
Select an EC2 key pair to securely log in to your EC2 instances. [Learn more](#)

Choose a key pair [Create key pair](#)

Cancel [Skip to review](#) [Previous](#) [Next](#)

1. Service access configuration

- For Service role, select **aws-elasticbeanstalk-service-role** created earlier
- For EC2 instance profile, select **aws-elasticbeanstalk-ec2-role** created earlier
- Leave the **EC2 key pair** empty as SSM Sessions Manager will be used to SSH into the private EC2 instances

2. Click **Next** to proceed.

Step 3: Set up Networking, database, and tags

1. Instance settings

- Choose the VPC named **Umarsatti-VPC** (created earlier)
- Public IP address: Disabled (default)

- Instance subnets:
 - Availability Zone: us-east-1a and us-east-1b
 - Subnets: Private-Subnet-A and Private-Subnet-B

2. Database settings: None

3. Tags: None

4. Click **Next** to proceed

The screenshot shows the 'Set up networking, database, and tags' step in the AWS CloudFormation console. The left sidebar shows the progress of the 'Create environment' wizard, with 'Set up networking, database, and tags' selected. The main content area is titled 'Set up networking, database, and tags - optional' and contains three sections: 'Instance settings', 'Database', and 'Tags'.

Instance settings

Choose a subnet in each AZ for the instances that run your application. To avoid exposing your instances to the Internet, run your instances in private subnets and load balancer in public subnets. To run your load balancer and instances in the same public subnets, assign public IP addresses to the instances. [Learn more](#)

VPC
Launch your environment in a custom VPC instead of the default VPC. You can create a VPC and subnets in the VPC management console. [Learn more](#)

vpc-0d567942b0866f7d | (172.20.0.0/16) | Umarsatt-VPC [Create VPC](#)

Public IP address
Assign a public IP address to the Amazon EC2 instances in your environment.
☐ Enable

Instance subnets

Filter instance subnets

Availability Zone	Subnet	CIDR	Name
☐ us-east-1a	subnet-04c40e700692e35a	172.20.10.0/24	Public-Subnet-A
☐ us-east-1b	subnet-05596a689c10db796	172.20.15.0/24	Public-Subnet-B
☒ us-east-1b	subnet-08bc5bb15336bd843	172.20.25.0/24	Private-Subnet-B
☒ us-east-1a	subnet-0cf4da2d55672111	172.20.20.0/24	Private-Subnet-A

Database
Integrate an RDS SQL database with your environment. [Learn more](#)

☐ Enable database

Tags
Apply up to 50 tags. You can use tags to group and filter your resources. A tag is a key-value pair. The key must be unique within the resource and is case-sensitive. [Learn more](#)

Buttons: Cancel, Skip to review, Previous, Next

Step 4: Configure instance traffic and scaling

1. Instances

- Root volume (boot device)
 - **Root volume type:** General Purpose SSD (gp2)
 - **Size:** 10 GB
- Amazon CloudWatch monitoring
 - **Monitoring interval:** 1 minute
- Instance metadata service (IMDS)
 - **IMDSv1:** Disabled
- EC2 Security groups
 - Select **EC2-SG** (created in Task 1.8)

2. Capacity

- Autoscaling group
 - Environment type: Load balanced
 - Minimum instances: 1
 - Maximum instances: 2
 - Fleet composition: On-Demand instances

- Architecture: x86_64
- Instance types: t3.micro and t3.small
- AMI ID: ami-054b161ba0194ab9e (default option)
- Scaling cooldown: 60 seconds
- Scaling triggers
 - Metric: CPUUtilization
 - Statistic: Maximum
 - Unit: Percentage
 - Period: 1 minute
 - Breach duration: 2 minute
 - Upper threshold: 70%
 - Scale up increment: 1 EC2 instance
 - Lower threshold: 30%
 - Scale down increment: -1 EC2 instance

Scaling triggers

Metric
Change the metric that is monitored to determine if the environment's capacity is too low or too high.

CPUUtilization ▼

Statistic
Choose how the metric is interpreted.

Maximum ▼

Unit

Percent ▼

Period
The period between metric evaluations.

1 Min

Breach duration
The amount of time a metric can exceed a threshold before triggering a scaling operation.

2 Min

Upper threshold

70

Scale up increment

1 EC2 instances

Lower threshold

30 capacity

Scale down increment

-1 EC2 instances

3. Load balancer network settings

- Visibility: Public
- Load balancer subnets: Public-Subnet-A and Public-Subnet-B

4. Load balancer type

- Application Load Balancer
- Dedicated

5. Listeners

- Listener Port: 80

- Listener Protocol: HTTP
- Enabled: Yes

6. Rules: None

7. Log files access: Disabled

Note: Log files access can be enabled to store logs captured by Load balancer into Amazon S3 for further analysis.

8. Click **Next** to proceed

Step 5: Configure updates, monitoring, and logging

1. Monitoring

- Health reporting and event streaming
 - System: Enhanced Monitoring
 - CloudWatch Custom Metrics – Instance
 - Select custom metrics such as ApplicationRequests or InstanceHealth
 - CloudWatch Custom Metrics – Environment
 - Select custom metrics such as InstanceWarning or ApplicationRequests
- Health event streaming to CloudWatch Logs
 - Log streaming: Enabled
 - Retention: 7 days
 - Lifecycle: Keep logs after environment termination

2. Managed platform updates:

- Managed updates: Disabled

3. Email notifications:

- Email: umarsatti.15@gmail.com

4. Rolling updates and deployments

- Application deployments
 - Deployment policy: All at once (only for development environment)
- Configuration updates
 - Rolling update type: Disabled
- Deployment preferences
 - Ignore health check: False
 - Health threshold: Ok
 - Command timeout: 600

5. Platform software

- Container options

- Proxy server: Nginx
- Amazon X-Ray
 - X-ray daemon: Disabled
- S3 log storage
 - Rotate logs: Disabled
- Instance log streaming to CloudWatch logs
 - Log streaming: Enabled
 - Retention: 7 days
 - Lifecycle: Keep logs after termination
- Environment properties: None

6. Click **Next** to proceed to the final step

Step 6: Review

- Review the whole configuration
- Apply changes by clicking the **Edit** button
- Proceed to create the Elastic Beanstalk environment

Review Info

Step 1: Configure environment Edit

Environment information

Environment tier	Application name
Web server environment	nodejs-application
Environment name	Application code
nodejs-env	Sample application
Platform	
arn:aws:elasticbeanstalk:us-east-1::platform/Node.js 22 running on 64bit Amazon Linux 2023/6.7.0	

Step 2: Configure service access Edit

Service access Info

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely log in to your EC2 instances.

Service role	EC2 instance profile
arn:aws:iam::730335208305:role/aws-elasticbeanstalk-service-role	aws-elasticbeanstalk-ec2-role

Step 3: Set up networking, database, and tags Edit

Networking, database, and tags Info

Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that's integrated with your environment.

Network

VPC	Public IP address	Instance subnets
vpc-0d367942eb866ff7d	false	subnet-08bc5bb15336bd043,subnet-0cf4dda2d55672111

Tags

Key	Value
No tags	
There are no tags defined	

Step 4: Configure instance traffic and scaling

Edit

Instance traffic and scaling [Info](#)

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances

Root volume type
gp2

Instance size
10

IMDSv1
Disabled

EC2 Security Groups
sg-035c1d8003d17fd23

Capacity

Environment type
Load balanced

Min instances
1

Max instances
2

Fleet composition
On-Demand instances

On-demand base
0

On-demand above base
70

Capacity rebalancing
Disabled

Scaling cooldown
60

Processor type
x86_64

Instance types
t3.micro,t3.small

AMI ID
ami-054b161ba0194ab9e

Availability Zones
Any

Metric
CPUUtilization

Statistic
Maximum

Unit
Percent

Period
1

Breach duration
2

Upper threshold
70

Scale up Increment
1

Lower threshold
30

Scale down Increment
-1

Load balancer

Load balancer visibility
public

Load balancer subnets
subnet-05596a689c10db796,subnet-04c40e7006692e33a

Load balancer type
application

Load balancer is shared
false

Store logs
Disabled

Step 5: Configure updates, monitoring, and logging

Edit

Updates, monitoring, and logging [Info](#)

Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring

System
enhanced

Cloudwatch custom metrics - instance
—

Cloudwatch custom metrics - environment
—

Log streaming
Enabled

Retention
7

Lifecycle
false

Updates

Managed updates
Disabled

Deployment batch size
100

Deployment batch size type
Percentage

Command timeout
600

Deployment policy
AllAtOnce

Health threshold
Ok

Ignore health check
false

Instance replacement
false

Notifications email
umarsatti.15@gmail.com

Platform software

Lifecycle
false

Log streaming
Enabled

Proxy server
nginx

Logs retention
7

Rotate logs
Disabled

Update level
minor

X-Ray enabled
Disabled

Environment properties

Source



Key



Value



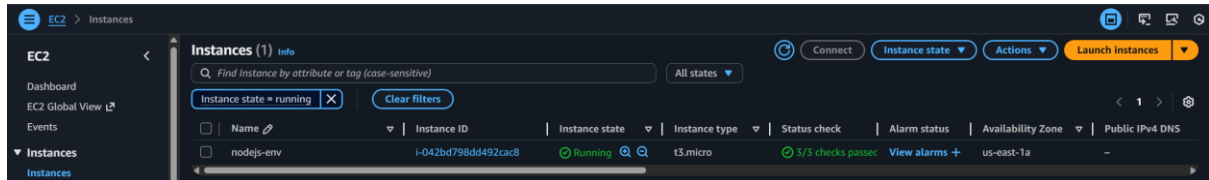
No environment properties
There are no environment properties defined

Task 3: Verify Infrastructure Deployment

Verify EC2 Instances are launched

Steps:

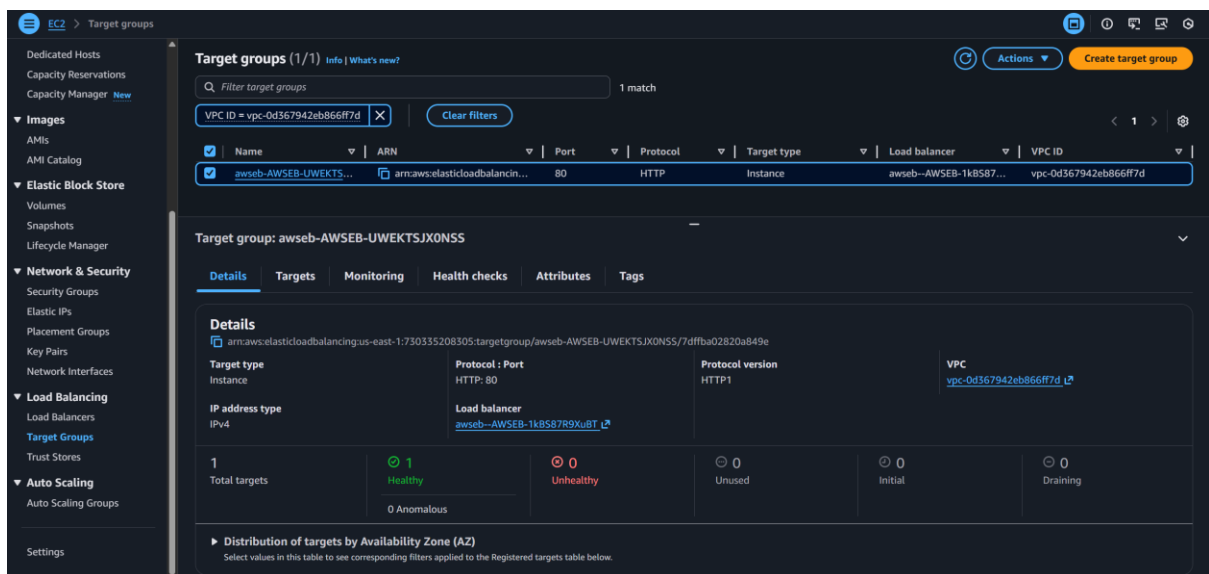
1. Navigate to the EC2 console using the search bar above.
2. Click Instances in the left navigation bar.
3. Verify that the EC2 instance is launched and running.



Verify Target Groups

Steps:

1. In the EC2 console, scroll down the navigation bar.
2. Choose the Target Groups option located in the left navigation bar and click on it.
3. Select the specific target group and in the Details section, make sure the target instance is Healthy.

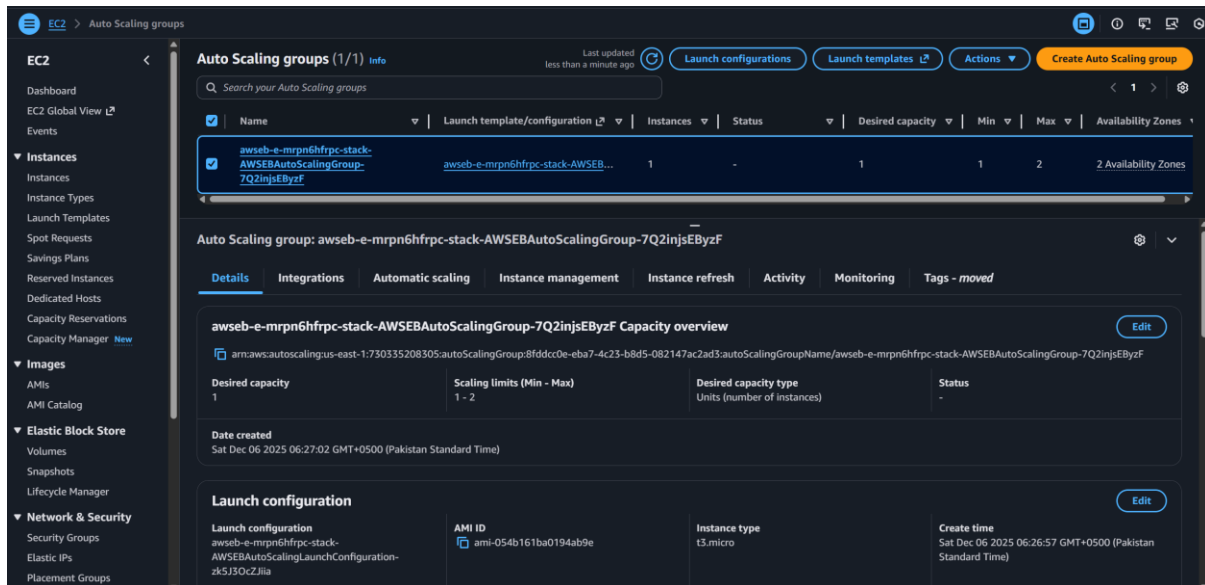


Verify Autoscaling Group

Steps:

1. In the EC2 console, choose the Auto Scaling groups option located in the left navigation bar and click on it.

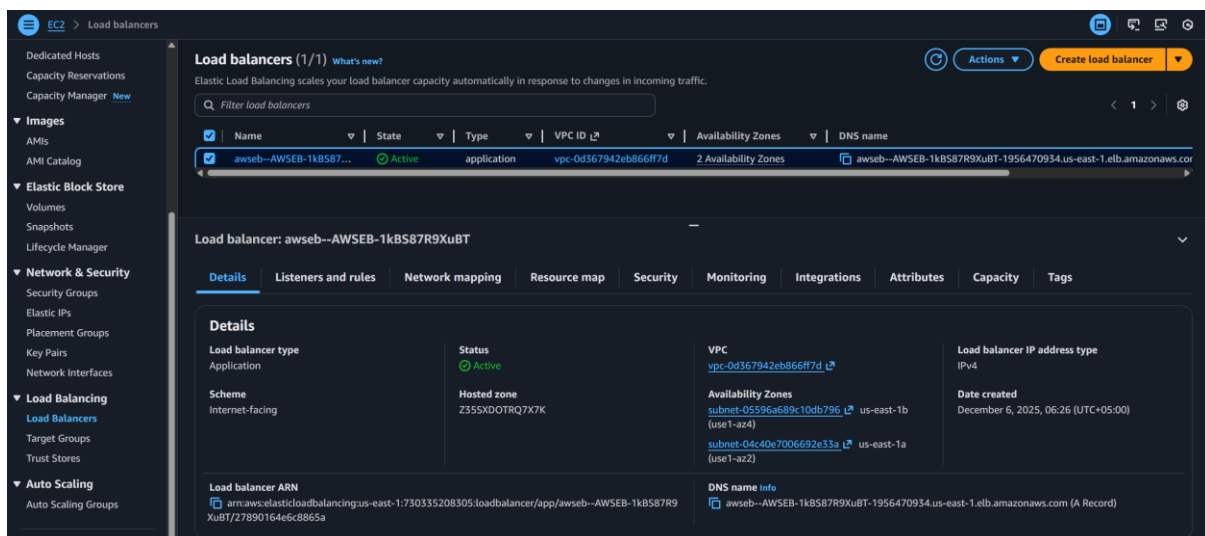
2. Select the specific Auto Scaling group created by the Elastic Beanstalk environment.
3. In the Details section, make sure the capacity settings are correct.



Verify ALB is working

Steps:

1. In the EC2 console, choose the Load Balancers option located in the left navigation bar and click on it.
2. Select the specific Load Balancer created by the Elastic Beanstalk environment.
3. In the Details section, make sure the Application Load Balancer status is Active and has the correct VPC settings.



Verify if application is running using Elastic Beanstalk URL

- *Nodejs-env-eba0ugqtyv3m.us-east-1.elasticbeanstalk.com*

The image shows two parts of a web application deployment. The top part is a screenshot of the AWS Elastic Beanstalk console. It displays the 'nodejs-env' environment overview, including its health status (Ok), environment ID (e-mrpn6hfrpc), application name (nodejs-application), and platform (Node.js 22 running on 64bit Amazon Linux 2023/6.7.0). The bottom part is a screenshot of the application's web page, which displays a 'Simple Node.js Application' title and a message about deploying a Node.js app to AWS Elastic Beanstalk.

Amazon SNS

1. Subscribe to the SNS Notification

AWS Notification - Subscription Confirmation ➤ Inbox x



AWS Notifications <no-reply@sns.amazonaws.com>
to me ▾

You have chosen to subscribe to the topic:
arn:aws:sns:us-east-1:730335208305:ElasticBeanstalkNotifications-Environment-nodejs-env

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):
[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)

↩ Reply

➡ Forward





Simple Notification Service

Subscription confirmed!

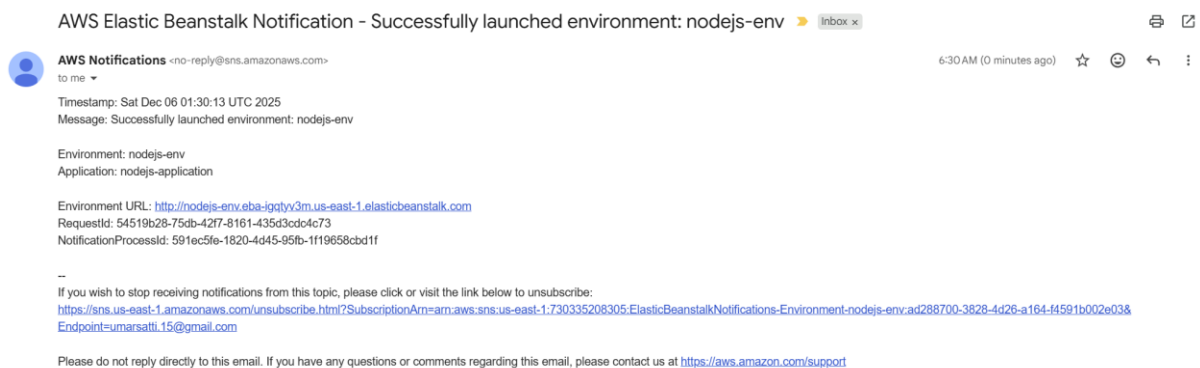
You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-1:730335208305:ElasticBeanstalkNotifications-Environment-nodejs-env:ad288700-3828-4d26-a164-f4591b002e03

If it was not your intention to subscribe, [click here to unsubscribe](#).

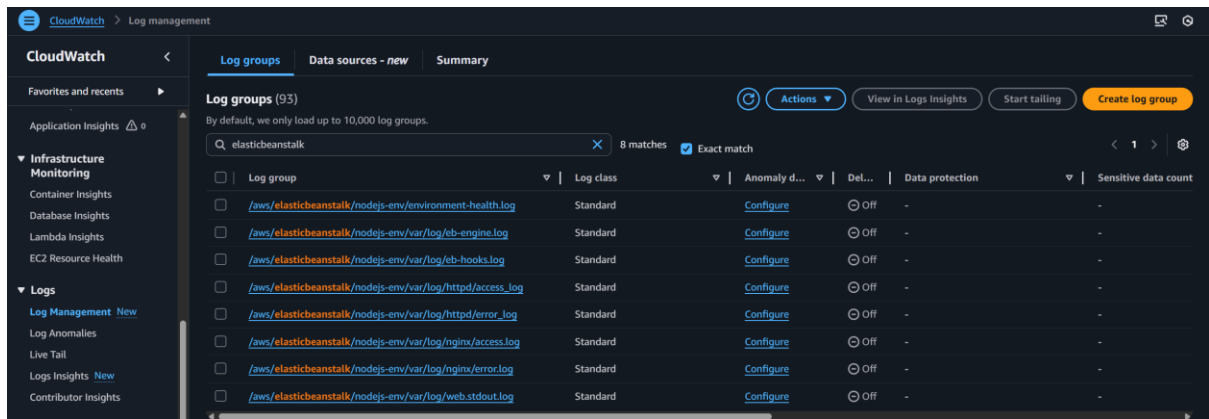
2. Receive notifications when Elastic Beanstalk successfully deploys application



CloudWatch Log groups

Steps:

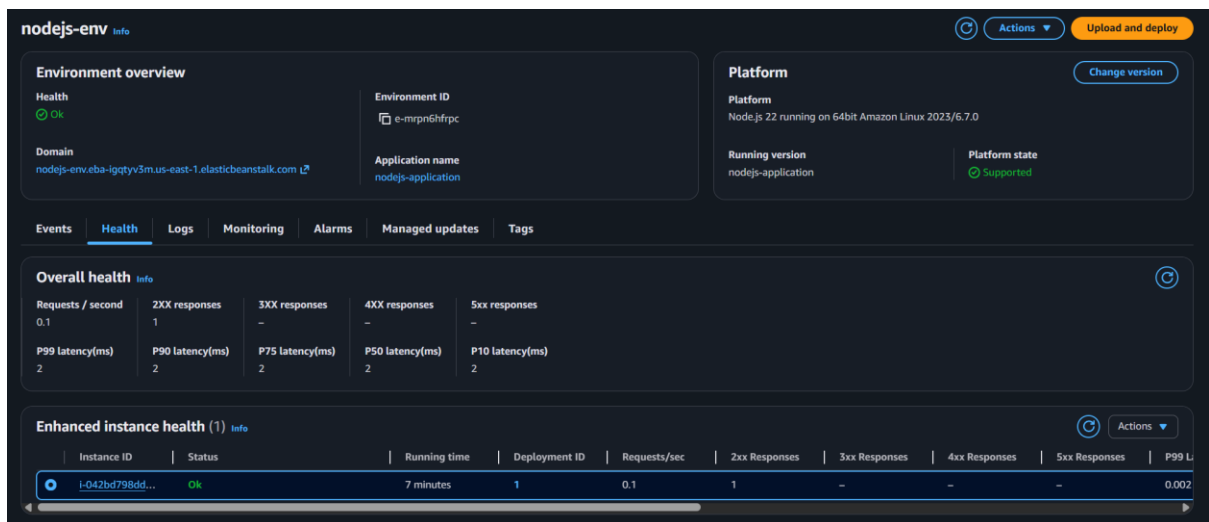
1. Sign in to the AWS Management Console.
2. Navigate to **CloudWatch** service using the search bar at the top.
3. In the CloudWatch console, click **Log Management** located in the left navigation bar under **Logs**.
4. Search elasticbeanstalk/nodejs-env to display a list of log groups associated with the Elastic Beanstalk environment.
5. These log groups display NGINX, Apache, and Beanstalk environment related logs.



Environment Health

Steps:

1. Go to Elastic Beanstalk console using the search bar.
2. In the console, click **Environments**.
3. Choose the environment created earlier named nodejs-env.
4. View the **Health** tab.
5. This displays all the custom metrics chosen during environment creation steps.
Examples include HTTP responses (2xx, 3xx, 4xx etc.), latency, CPU metrics and more.



CloudWatch Alarms

Steps:

1. Sign in to the AWS Management Console.
2. Navigate to CloudWatch service using the search bar at the top.

3. In the CloudWatch console, click **All alarms** located in the left navigation bar under **Alarms**.
4. View the state of the Alarms that were configured during Elastic Beanstalk environment creation.

CloudWatch Alarm Conditions

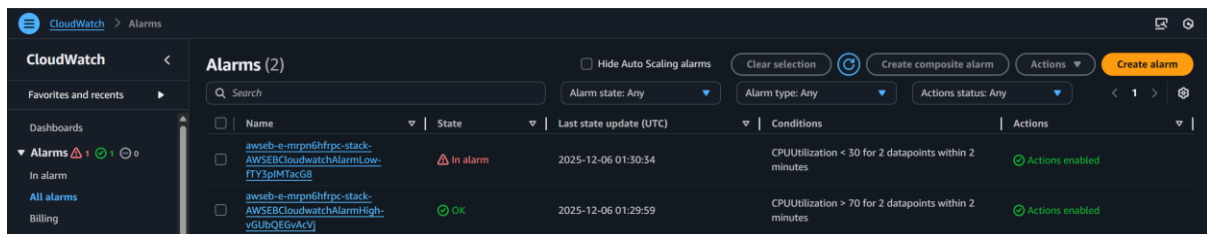
Elastic Beanstalk automatically created two CloudWatch alarms that monitor the CPU utilization of the EC2 instances in the environment:

1. Scale-Down Alarm

- **Condition:** CPUUtilization < **30%** for **2 consecutive datapoints** within **2 minutes**
- **Purpose:** Detects when the environment is under low load.
- **Action:** Triggers the Auto Scaling policy to **reduce capacity** (scale down the number of EC2 instances).
- **Current State:** **IN ALARM**, meaning CPU is currently below the 30% threshold.

2. Scale-Up Alarm

- **Condition:** CPUUtilization > **70%** for **2 consecutive datapoints** within **2 minutes**
- **Purpose:** Detects high load or increased demand.
- **Action:** Triggers the Auto Scaling policy to **add an EC2 instance** (scale up).
- **Current State:** **OK**, meaning CPU is not above 70% at this moment.

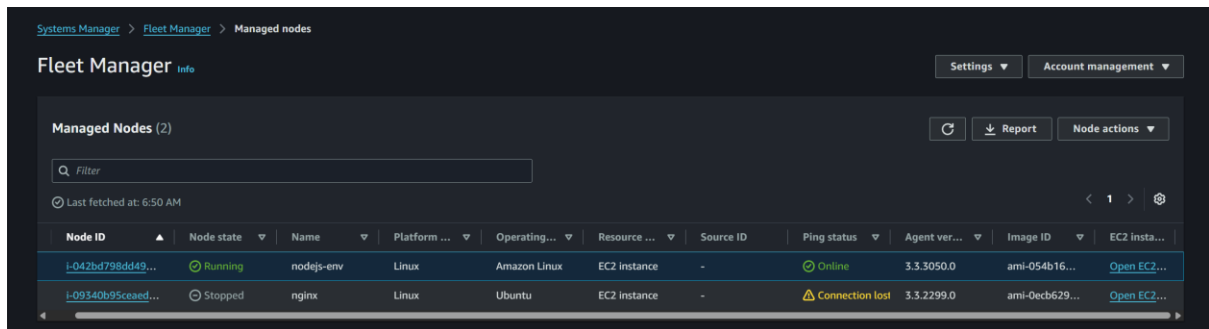


Name	State	Last state update (UTC)	Conditions	Actions
awseb-e-mrpn6hfrpc-stack-AWSEBCloudwatchAlarmLow-FTY3pIMTAcG8	In alarm	2025-12-06 01:30:34	CPUUtilization < 30 for 2 datapoints within 2 minutes	Actions enabled
awseb-e-mrpn6hfrpc-stack-AWSEBCloudwatchAlarmHigh-vGLubQE6vAcVj	OK	2025-12-06 01:29:59	CPUUtilization > 70 for 2 datapoints within 2 minutes	Actions enabled

Using CloudWatch Alarms

Step 1: Verify SSM agent is installed on EC2

- Go to Systems Manager
- Select Fleet Manager tab located on the left
- Verify that EC2 named **nodejs-env** is present and running



Step 2: SSH into EC2 instance

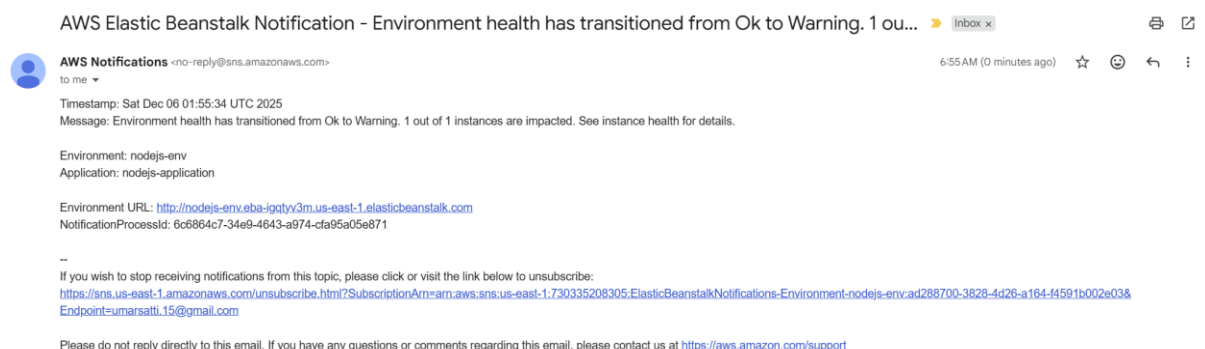
- Go to EC2 Console and select the EC2 instance
- Click Connect located on the top right
- Navigate to Session Manager tab and click Connect

Step 3: Using stress utility to increase CPU usage on this instance

- Install stress utility using:
 - `sudo yum install stress-ng -y`
- Run stress command
 - `stress-ng --cpu 0 --cpu-load 90 --timeout 360`

```
sh-5.2$ stress-ng --cpu 0 --cpu-load 90 --timeout 360
stress-ng: info:  [29023] setting to a 360 second (6 mins, 0.00 secs) run per stressor
stress-ng: info:  [29023] dispatching hogs: 2 cpu
```

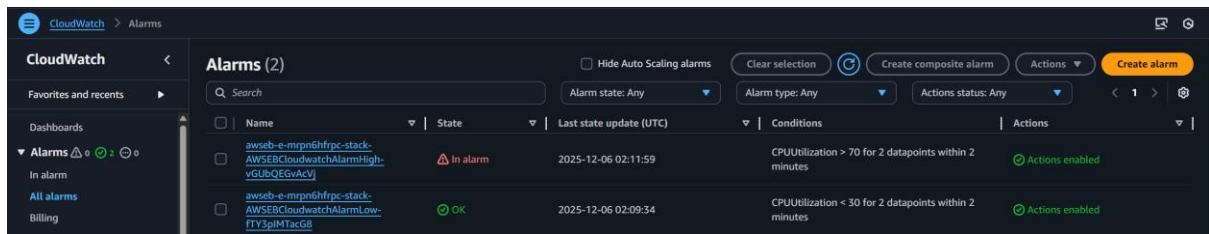
- This stress command is used to increase the CPU usage of EC2 instance up to 90%.
- Notifications are sent to the Email regarding Environment health and status of EC2 instances



Step 4: Verify CloudWatch Alarms

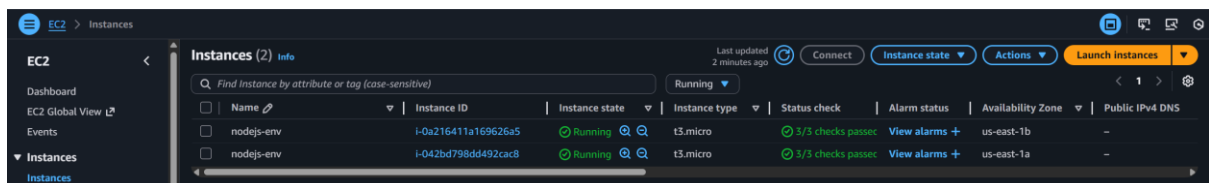
- Go to CloudWatch console
- Select **All alarms** under **Alarms** tab

- It should display the 70% CPUUtilization condition in **ALARM** state and transition the 30 % CPUUtilization condition in OK state because the CPU usage is now around 90% (greater than 70%).



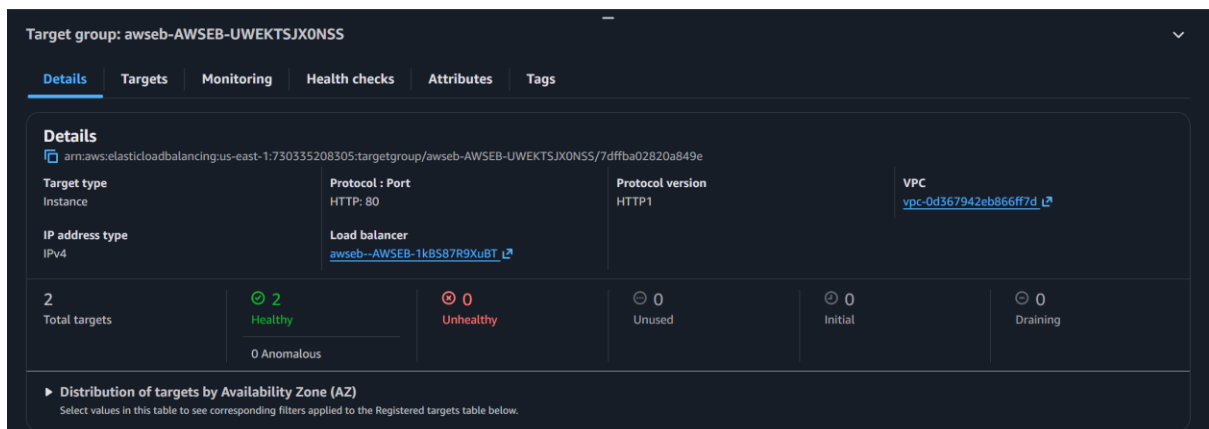
Step 5: Confirm EC2 creation

- Navigate to the EC2 console using the search bar
- Choose the **Instances** option in the left navigation bar
- A new instance should be deployed as shown in the image below



Step 6: Target groups

- Navigate to the Target groups within the EC2 console
- Confirm that newly launched instance has been registered as a **Healthy** target



Step 7: Elastic Beanstalk events

- The scaling events caused by CloudWatch Alarms produced events in Elastic Beanstalk **Events** section
- Indicates the Environment health transitioned from Ok to Degraded
- An instance was added once the CPUUtilization went up (above 70%).
- The instance was terminated once the CPUUtilization went down (below 70%).

December 6, 2025 07:19:32 (UTC+5)	INFO	Removed instance [i-0a216411a169626a5] from your environment.
December 6, 2025 07:19:32 (UTC+5)	INFO	Environment health has transitioned from Degraded to Ok. 66.7 % of the requests to the ELB are erroring with HTTP 4xx. Insufficient request rate (1.5 requests/min) to determine application health (7 minutes ago).
December 6, 2025 07:18:32 (UTC+5)	WARN	Environment health has transitioned from Ok to Degraded. ELB processes are not healthy on 1 out of 2 instances. 66.7 % of the requests to the ELB are erroring with HTTP 4xx. Insufficient request rate (1.5 requests/min) to determine application health (6 minutes ago). ELB health is failing or not available for 1 out of 2 instances.
December 6, 2025 07:14:33 (UTC+5)	INFO	Environment health has transitioned from Warning to Ok.
December 6, 2025 07:13:33 (UTC+5)	INFO	Added instance [i-0a216411a169626a5] to your environment.

Task 4: Update Application Code and Redeploy

This task involves making changes to the Node.js application and deploying the updated version to the existing Elastic Beanstalk environment.

4.1 Prepare Updated Application Code

Make any required modifications to the Node.js project (for example, updating index.html, adding routes, or changing text).

After making these changes:

1. Ensure the project contains the files: **app.js**, **package.json**, and **public/** folder
2. Create a ZIP file of the application.

Important: Files must be at the **root level** of the ZIP, not inside a parent folder.

Upload and deploy

To deploy a previous version, go to the [Application versions page](#)

Upload application

Choose file

File name: nodejs-app-updated.zip
File must be less than 500MB max file size

Version label
Unique name for this version of your application code.
nodejs-application-1

► **Deployment preferences**

The application version will be deployed using the Rolling policy.

Current number of EC2 instances: 1

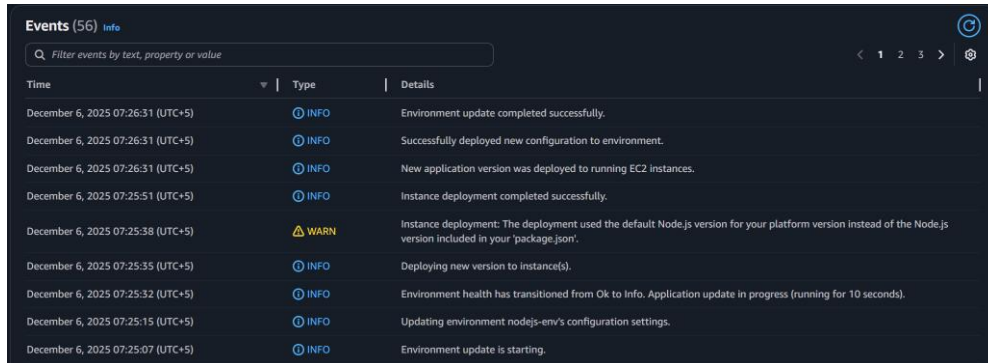
Cancel Deploy

4.2 Upload Updated Code in Elastic Beanstalk

- Open the **AWS Elastic Beanstalk console**.
- Select the **application** you previously created.
- Open the **environment** named nodejs-env.
- Click **Upload and Deploy** on the top right.
- Choose the ZIP file containing your updated application code.
- Provide a version label (optional), then click **Deploy**.

Elastic Beanstalk will automatically handle:

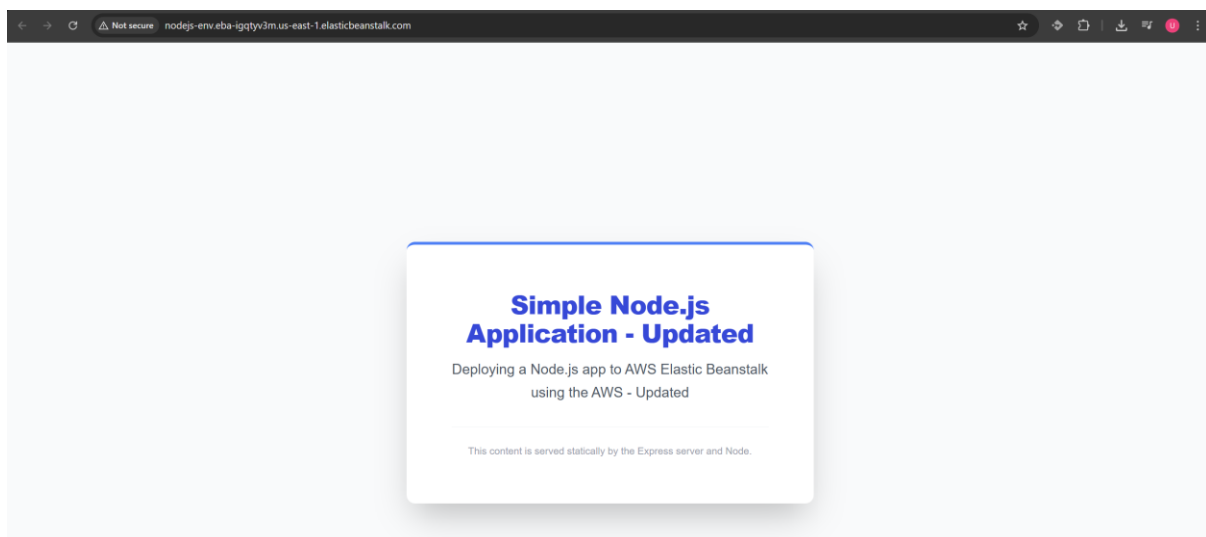
- Uploading the version
- Rebuilding the environment
- Deploying the new code to EC2 instances



Time	Type	Details
December 6, 2025 07:26:31 (UTC+5)	INFO	Environment update completed successfully.
December 6, 2025 07:26:31 (UTC+5)	INFO	Successfully deployed new configuration to environment.
December 6, 2025 07:26:31 (UTC+5)	INFO	New application version was deployed to running EC2 instances.
December 6, 2025 07:25:51 (UTC+5)	INFO	Instance deployment completed successfully.
December 6, 2025 07:25:38 (UTC+5)	WARN	Instance deployment: The deployment used the default Node.js version for your platform version instead of the Node.js version included in your 'package.json'.
December 6, 2025 07:25:35 (UTC+5)	INFO	Deploying new version to instance(s).
December 6, 2025 07:25:32 (UTC+5)	INFO	Environment health has transitioned from Ok to Info. Application update in progress (running for 10 seconds).
December 6, 2025 07:25:15 (UTC+5)	INFO	Updating environment nodejs-env's configuration settings.
December 6, 2025 07:25:07 (UTC+5)	INFO	Environment update is starting.

4.3 Validate the Deployment

- After successful deployment, test the application using Elastic Beanstalk URL:
 - ***nodejs-env.eba-igqtyv3m.us-east-1.elasticbeanstalk.com***
- Open this URL in the browser to confirm that your changes are reflected in the deployed application.



Task 5: Clean Up

The final step is to remove the AWS resources to avoid unnecessary charges.

5.1 Terminate the Elastic Beanstalk Environment

1. Go to the **Elastic Beanstalk console**.
2. Click **Environments** from the left sidebar.
3. Select the environment named **nodejs-env**.
4. Click the **Actions** button (top right).
5. Choose **Terminate environment**.

Elastic Beanstalk will:

- Terminate the EC2 instances
- Remove the load balancer
- Delete Auto Scaling resources
- Clean up associated health checks, alarms, and configuration

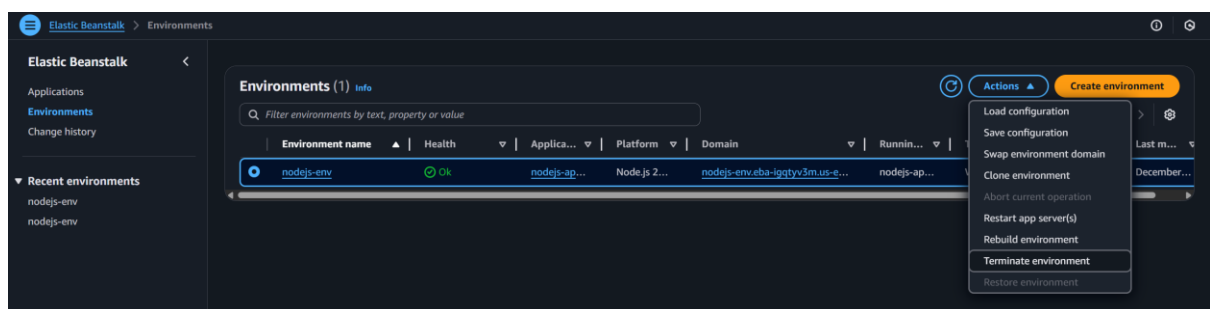
Note: Termination may take several minutes.

5.2 Optional: Delete the Application

If the application is no longer required:

1. Return to the Elastic Beanstalk **Applications** page.
2. Select your application (nodejs-application).
3. Click **Actions** and then click **Delete Application**.

This completes the cleanup process and prevents any further AWS charges.



Events (78) [info](#)

Filter events by text, property or value

< 1 2 3 4 > 

Time	Type	Details
December 6, 2025 07:34:40 (UTC+5)	INFO	terminateEnvironment completed successfully.
December 6, 2025 07:34:38 (UTC+5)	INFO	Deleting SNS topic for environment nodejs-env.
December 6, 2025 07:34:37 (UTC+5)	INFO	Deleted security group named: sg-00072e8598cd20c32
December 6, 2025 07:34:37 (UTC+5)	INFO	Deleted security group named: sg-097e39570e731041f
December 6, 2025 07:34:37 (UTC+5)	INFO	Deleted Auto Scaling launch configuration named: awseb-e-mrpn6hfrpc-stack-AWSEBAutoScalingLaunchConfiguration-zk5J3QcZJia
December 6, 2025 07:34:37 (UTC+5)	INFO	Deleted target group named: arn:aws:elasticloadbalancing:us-east-1:730335208305:targetgroup/awseb-AWSEB-UWEKTSJXONSS/7dffba02820a849e
December 6, 2025 07:34:31 (UTC+5)	INFO	Removed instance [i-042bd798dd492cac8] from your environment.
December 6, 2025 07:34:22 (UTC+5)	INFO	Deleted Auto Scaling group named: awseb-e-mrpn6hfrpc-stack-AWSEBAutoScalingGroup-7Q2InjsEByzf
December 6, 2025 07:32:35 (UTC+5)	INFO	Deleted load balancer named: arn:aws:elasticloadbalancing:us-east-1:730335208305:loadbalancer/app/awseb--AWSEB-1kBS87R9XuBT/27890164e6c8865a
December 6, 2025 07:31:34 (UTC+5)	INFO	Waiting for EC2 instances to terminate. This may take a few minutes.
December 6, 2025 07:31:34 (UTC+5)	INFO	Deleted Auto Scaling group policy named: arn:aws:autoscaling:us-east-1:730335208305:scalingPolicy:4011f23f-1ed6-4612-81bd-830b215a01e0:autoScalingGroupName/awseb-e-mrpn6hfrpc-stack-AWSEBAutoScalingGroup-7Q2InjsEByzf:policyName/awseb-e-mrpn6hfrpc-stack-AWSEBAutoScalingScaleDownPolicy-WL3oHYiBwR1F
December 6, 2025 07:31:34 (UTC+5)	INFO	Deleted Auto Scaling group policy named: arn:aws:autoscaling:us-east-1:730335208305:scalingPolicy:61c6ea06-31c0-4bda-8cc6-aea36509dd6c:autoScalingGroupName/awseb-e-mrpn6hfrpc-stack-AWSEBAutoScalingGroup-7Q2InjsEByzf:policyName/awseb-e-mrpn6hfrpc-stack-AWSEBAutoScalingScaleUpPolicy-k5WT97jcZtG
December 6, 2025 07:31:34 (UTC+5)	INFO	Deleted Load Balancer listener named: arn:aws:elasticloadbalancing:us-east-1:730335208305:listener/app/awseb--AWSEB-1kBS87R9XuBT/27890164e6c8865a/668f8a375dffcf470
December 6, 2025 07:31:34 (UTC+5)	INFO	Deleted CloudWatch alarm named: awseb-e-mrpn6hfrpc-stack-AWSEBCloudwatchAlarmLow-FTY3piMTacG8
December 6, 2025 07:31:34 (UTC+5)	INFO	Deleted CloudWatch alarm named: awseb-e-mrpn6hfrpc-stack-AWSEBCloudwatchAlarmHigh-vGUbQEGvAcVj
December 6, 2025 07:31:31 (UTC+5)	INFO	Environment health has transitioned from Ok to Info. Terminate in progress (running for 5 seconds).

Task 6: Troubleshooting

The following issues were encountered during deployment, testing, and scaling of the Elastic Beanstalk environment. Each issue includes the observed error, root cause, and exact steps taken to resolve it.

Issue 1: Elastic Beanstalk IAM Role Error

Problem Description:

When configuring IAM Role for the Elastic Beanstalk environment, an error occurred.

Select a service role that has the 'AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy' managed policy.

Root Cause:

Elastic Beanstalk was configured to use the default **service-linked role** (AWSServiceRoleForElasticBeanstalk), which does **not** include the required policy for **Managed Updates** (an optional feature but auto-selected). Elastic Beanstalk only allows Managed Platform Updates when using a **customer-managed service role** with the correct permissions.

Solution:

Create a custom IAM role named **aws-elasticbeanstalk-service-role** and attach the following managed policies:

- *AWSElasticBeanstalkEnhancedHealth*
- *AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy*

Then update the environment configuration to use this new service role. After assigning the custom role, Managed Updates can be enabled successfully and the user can move on to the next step.

Issue 2: CloudWatch Alarms Did Not Immediately Trigger During CPU Stress Test

Problem Description:

During testing of Auto Scaling using the stress and stress-ng utilities on the EC2 instance, CloudWatch alarms initially did not transition to **In Alarm** state, and scaling actions did not trigger immediately.

Root Cause:

CloudWatch metrics are collected at **1-minute intervals**, and alarm evaluation requires:

- **2 datapoints**
- **across 2 consecutive periods**

This introduces a **2–3 minute delay** before alarms trigger. Additionally, initial CPU spikes may not breach the threshold long enough to satisfy the evaluation window.

Solution:

- Increased the duration of the stress test:
- `stress-ng --cpu 0 --cpu-load 95 --timeout 300`
- Ensured the CPU load remained above the **70% scale-up threshold** for multiple minutes.

Once the CPU remained consistently high, the “High CPU” alarm transitioned to **In Alarm** state, and a new EC2 instance was launched successfully.

Issue 3: stress-ng Failed With “temp-path '.' must be readable and writable”**Problem Description:**

Running stress-ng on the EC2 instance returned:

- *aborting: temp-path '.' must be readable and writable*

Root Cause:

The command was executed inside a restricted directory where the user did not have write permissions. The **stress-ng** utility requires the ability to write temporary files during execution.

Solution:

Change to a writable directory such as the home directory or /tmp before running `cd ~` OR `cd /tmp` to move to the correct directory and then running:

- `stress-ng --cpu 0 --cpu-load 95 --timeout 300`

This resolved the permission issue and allowed the stress test to run properly.

Issue 4: Logs Not Showing in the Elastic Beanstalk Console**Problem Description:**

Under the **Logs** tab, no log files appeared even after deployment.

Root Cause:

Elastic Beanstalk does not automatically pull logs unless a request is made. Logs are stored directly on the EC2 instance and must be manually retrieved.

Solution:

Use the **Request Logs** button in the EB console:

- **Request last 100 lines** or
- **Request full logs**

Once triggered, Elastic Beanstalk fetches the logs temporarily, and they appear in the console or can be downloaded as a ZIP folder. This behavior is normal for EB-managed environments.